

Reviewer Pack — Matroid Rank Lower-Bounds Disjointness Communication Complex...

Entry 13efcfb2aa35 · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-05-08 06:23:02 UTC

Matroid Rank Lower-Bounds Disjointness Communication Complexity

Entry ID: 13efcfb2aa35 **Verdict:** INCONCLUSIVE **Recorded:** 2026-05-08 06:23:02 UTC

1. Conjecture

Field A (mathematical branch): Matroid Theory **Field B** (complexity object): Randomized Communication Complexity of DISJOINTNESS

Statement:

For any matroid M representable over $\text{GF}(2)$, the communication complexity $R_{1/3}(\text{DISJ})$ of the characteristic function of M 's independent sets satisfies $R_{1/3}(\text{DISJ}) \geq \text{rank}(M) \cdot \log_2(n)$.

Rationale (proposer's reasoning):

Matroid rank captures combinatorial structure of independence, while DISJOINTNESS requires coordination across subsets. Linking them via $\text{GF}(2)$ representability exposes algebraic constraints on communication protocols.

Taxonomy category: LIFTING (status at proposal time: alive)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): 921cfbf76cf27918

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

default: $\geq 80\%$ seeds must support, no counterexample

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	SAFE	0.00	UNCERTAIN	UNCERTAIN
NATURAL_PROOFS	SAFE	0.00	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	SAFE	0.00	UNCERTAIN	UNCERTAIN
KARP_LIPTON	SAFE	0.95	SAFE	SAFE

4. Novelty audit

Verdict: NOVEL against 0 hits across arXiv + Semantic Scholar + ECCC

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, ≤ 90 s wall time per cycle **Execution:** rc=1, elapsed=0.1s

5.1 Generated Python source

```
import random
import math
from collections import defaultdict

def generate_random_matroid(n, p):
    A = [[random.choice([0, 1]) for _ in range(n)] for _ in range(n)]
    while not is_independent_set(A, n, p):
        A = [[random.choice([0, 1]) for _ in range(n)] for _ in range(n)]
    return A

def is_independent_set(A, n, p):
    for i in range(n):
        if any(all(B[j][k] == A[i][k] for k in range(i)) for j in range(i)):
            return False
    return True

def rank_of_matroid(A, n):
    I = []
    for i in range(n):
        if all(A[i][j] == 0 for j in range(i) if i != j):
            I.append(i)
    return len(I)

def simulate_disjointness_protocol(A, n):
    depth = 0
    queue = [set()]
    while queue:
        new_queue = []
        for s in queue:
            if len(s) == n:
                break
            for i in range(n):
                if i not in s and all(A[i][j] == A[j][i] for j in s):
                    new_queue.append(s | {i})
        depth += 1
        if not new_queue:
            return float('inf')
        queue = new_queue
    return depth

def run_trial(seed: int) -> dict:
    random.seed(seed)
    n = 16
    p = 2
    A = generate_random_matroid(n, p)
```

```

rank = rank_of_matroid(A, n)
protocol_depth = simulate_disjointness_protocol(A, n)
metric_value = protocol_depth / (rank * math.log2(n))
conjecture_holds = metric_value >= 1
counterexample = "" if conjecture_holds else f"Protocol depth {protocol_depth} < {rank * math.log2(n)}"
return {
    "metric_name": "Communication Complexity",
    "metric_value": metric_value,
    "instances_tested": 1,
    "conjecture_holds": conjecture_holds,
    "counterexample": counterexample
}

if __name__ == "__main__":
    import sys
    seeds = [int(s) for s in sys.argv[1:]] or [2**i + 1 for i in range(5, 30)]
    results = []
    for seed in seeds:
        result = run_trial(seed)
        print(f"TRIAL: {result}")
        results.append(result)

    mean_value = sum(r["metric_value"] for r in results) / len(results)
    std_value = math.sqrt(sum((r["metric_value"] - mean_value)**2 for r in results) / len(results))
    support_fraction = sum(1 for r in results if r["conjecture_holds"]) / len(results)

    if all(r["conjecture_holds"] for r in results):
        print(f"RESULT: SUPPORTED mean={mean_value} std={std_value} support_fraction={support_fraction}")
    elif support_fraction >= 0.8:
        print(f"RESULT: SUPPORTED mean={mean_value} std={std_value} support_fraction={support_fraction}")
    else:
        first_failing_seed = next(r["seed"] for r in results if not r["conjecture_holds"])
        print(f"RESULT: FALSIFIED counterexample={results[0]['counterexample']} first_failing_seed={first_failing_seed}")

```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

--STDERR--

Traceback (most recent call last):

```

File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_e1083e7e.py", line 76, in <module>
    result = run_trial(seed)
    ~~~~~^~~~~~

```

```

File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_e1083e7e.py", line 57, in run_trial
    A = generate_random_matroid(n, p)
    ~~~~~^~~~~~

```

```

File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_e1083e7e.py", line 19, in generate_random_matroid
    while not is_independent_set(A, n, p):
    ~~~~~^~~~~~

```

```

File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_e1083e7e.py", line 25, in is_independent_set
    if any(all(B[j][k] == A[i][k] for k in range(i)) for j in range(i)):
    ~~~~~^~~~~~

```

```

File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_e1083e7e.py", line 25, in <genexpr>

```

```

    if any(all(B[j][k] == A[i][k] for k in range(i)) for j in range(i)):
        ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_e1083e7e.py", line 25, in <genexpr>
    if any(all(B[j][k] == A[i][k] for k in range(i)) for j in range(i)):
        ^
NameError: name 'B' is not defined

```

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

Test crashed due to undefined variable 'B' in `is_independent_set`, preventing any data collection | next: Fix the `NameError` in `is_independent_set` by properly initializing matrix B

11. Audit log (LLM calls)

Total LLM calls: 8

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	qwen3:8b	0	0	43294
2	propose	ollama_remote	qwen3:8b	0	0	30469
3	preregistration	ollama_remote	qwen3:8b	0	0	19960
4	novelty	ollama_remote	qwen3:8b	0	0	16533
5	novelty	ollama_remote	qwen3:8b	0	0	12830
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	18088
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	8860
8	judge	ollama_remote	qwen3:8b	0	0	14866

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 164901 ms total latency. Provider mix: {'ollama_remote': 8}

(full prompt+response transcripts available in `research/audit/13efcfb2aa35.jsonl`)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/13efcfb2aa35.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)

Replay tarball: `research/replay/13efcfb2aa35.tar.gz` (if generated)